TP2 - Intégration continue avec GitHub Actions

Durée : 2h

Prérequis : CM + TD1 + TD2 suivis, compte GitHub

Objectifs pédagogiques

- Configurer GitHub Actions pour un projet Symfony
- Mettre en place des tests automatisés
- Implémenter l'analyse de qualité de code
- Gérer le cache et optimiser les performances

Prérequis techniques

Setup initial

1. Fork du repository : `https://github.com/[prof]/symfony-contact-app`
2. Cloner votre fork en local
3. Créer une branche `develop` depuis `main`
4. Activer GitHub Actions dans Settings → Actions

Partie 1 : Premier workflow (25 min)

1.1 Création du workflow de base

Créez le fichier `.github/workflows/ci.yml` :

```yaml
```

```yaml
name: CI

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
    - uses: actions/checkout@v3

    - name: Setup PHP
      uses: shivammathur/setup-php@v2
      with:
        php-version: '8.2'
        extensions: mbstring, xml, ctype, iconv, intl, pdo, pdo_mysql, dom
        coverage: xdebug

    - name: Validate composer.json and composer.lock
      run: composer validate --strict

    - name: Install dependencies
      run: composer install --prefer-dist --no-progress

    - name: Run tests
      run: php bin/phpunit
```

1.2 Premier test

1. Committez et poussez le workflow
2. Allez sur GitHub → Actions
3. Observez l'exécution

📝 **Checkpoint 1** : Screenshot du premier build réussi

1.3 Ajout de la base de données

Modifiez le workflow pour ajouter MySQL :

```yaml
```

```yaml
jobs:
  test:
    runs-on: ubuntu-latest

    services:
      mysql:
        image: mysql:8.0
        env:
          MYSQL_ROOT_PASSWORD: root
          MYSQL_DATABASE: symfony_test
          MYSQL_USER: symfony
          MYSQL_PASSWORD: symfony
        ports:
          - 3306:3306
        options: --health-cmd="mysqladmin ping" --health-interval=10s --health-timeout=5s --health-retries=3

    steps:
    # ... steps précédents ...

    - name: Setup test database
      env:
        DATABASE_URL: mysql://symfony:symfony@127.0.0.1:3306/symfony_test
      run: |
        php bin/console doctrine:migrations:migrate --no-interaction
        php bin/console doctrine:fixtures:load --no-interaction --env=test
```

Partie 2 : Tests et couverture (30 min)

2.1 Configuration PHPUnit

Créez/modifiez `phpunit.xml.dist` :

```xml
```

```
<?xml version="1.0" encoding="UTF-8"?>
<phpunit xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:noNamespaceSchemaLocation="vendor/phpunit/phpunit/phpunit.xsd"
         colors="true"
         bootstrap="tests/bootstrap.php">

    <php>
        <ini name="display_errors" value="1" />
        <ini name="error_reporting" value="-1" />
        <server name="KERNEL_CLASS" value="App\Kernel" />
        <server name="APP_ENV" value="test" force="true" />
        <server name="APP_DEBUG" value="1" />
        <env name="SYMFONY_DEPRECATIONS_HELPER" value="disabled" />
    </php>

    <testsuites>
        <testsuite name="Project Test Suite">
            <directory>tests</directory>
        </testsuite>
    </testsuites>

    <coverage>
        <include>
            <directory>src</directory>
        </include>
    </coverage>
</phpunit>
```

2.2 Ajout d'un test

Créez `tests/Controller/ContactControllerTest.php` :

```
php
```

```php
<?php

namespace App\Tests\Controller;

use Symfony\Bundle\FrameworkBundle\Test\WebTestCase;

class ContactControllerTest extends WebTestCase
{
    public function testContactFormDisplay(): void
    {
        $client = static::createClient();
        $client->request('GET', '/contact');

        $this->assertResponseIsSuccessful();
        $this->assertSelectorTextContains('h1', 'Contact');
    }

    public function testContactFormSubmission(): void
    {
        $client = static::createClient();
        $client->request('GET', '/contact');

        $client->submitForm('Send', [
            'contact[name]' => 'John Doe',
            'contact[email]' => 'john@example.com',
            'contact[message]' => 'Test message'
        ]);

        $this->assertResponseRedirects('/contact/success');
    }
}
```

2.3 Couverture de code

Ajoutez au workflow :

```yaml
```

```yaml
    - name: Run tests with coverage
      env:
        DATABASE_URL: mysql://symfony:symfony@127.0.0.1:3306/symfony_test
      run: |
        php bin/phpunit --coverage-text --coverage-clover=coverage.xml

    - name: Upload coverage to Codecov
      uses: codecov/codecov-action@v3
      with:
        file: ./coverage.xml
        fail_ci_if_error: true
```

📝 **Checkpoint 2** : Taux de couverture obtenu : ___%

Partie 3 : Analyse de qualité (30 min)

3.1 PHPStan

Installez PHPStan :

```bash
composer require --dev phpstan/phpstan phpstan/phpstan-symfony
```

Créez `phpstan.neon` :

```neon
parameters:
    level: 6
    paths:
        - src
    symfony:
        container_xml_path: var/cache/dev/App_KernelDevDebugContainer.xml
```

Ajoutez au workflow :

```yaml
    - name: PHPStan Static Analysis
      run: vendor/bin/phpstan analyse
```

3.2 PHP CS Fixer

Installez PHP CS Fixer :

```bash
composer require --dev friendsofphp/php-cs-fixer
```

Créez `.php-cs-fixer.dist.php` :

```php
<?php

$finder = PhpCsFixer\Finder::create()
    ->in(__DIR__)
    ->exclude('var')
    ->exclude('vendor')
;

return (new PhpCsFixer\Config())
    ->setRules([
        '@Symfony' => true,
        'array_syntax' => ['syntax' => 'short'],
    ])
    ->setFinder($finder)
;
```

Ajoutez au workflow :

```yaml
    - name: PHP CS Fixer
      run: vendor/bin/php-cs-fixer fix --dry-run --diff
```

3.3 Security Check

Ajoutez au workflow :

```yaml
    - name: Security Check
      run: |
        symfony check:security
```

📝 **Checkpoint 3** : Résultats de l'analyse de qualité

Partie 4 : Optimisation avec cache (20 min)

4.1 Cache Composer

Modifiez le workflow :

```yaml
- name: Get Composer Cache Directory
  id: composer-cache
  run: echo "dir=$(composer config cache-files-dir)" >> $GITHUB_OUTPUT

- name: Cache Composer dependencies
  uses: actions/cache@v3
  with:
    path: ${{ steps.composer-cache.outputs.dir }}
    key: ${{ runner.os }}-composer-${{ hashFiles('**/composer.lock') }}
    restore-keys: |
      ${{ runner.os }}-composer-

- name: Install dependencies
  run: composer install --prefer-dist --no-progress
```

4.2 Cache Symfony

```yaml
- name: Cache Symfony
  uses: actions/cache@v3
  with:
    path: |
      var/cache
      public/build
    key: ${{ runner.os }}-symfony-${{ hashFiles('**/composer.lock') }}
```

4.3 Mesure des performances

Comparez les temps avant/après cache :

Étape	Sans cache	Avec cache	Gain
Composer install	___s	___s	___%
Build total	___s	___s	___%

Partie 5 : Matrice de tests (15 min)

5.1 Tests multi-versions

Modifiez le workflow pour tester plusieurs versions :

```yaml
jobs:
  test:
    runs-on: ubuntu-latest

    strategy:
      fail-fast: false
      matrix:
        php-version: ['8.1', '8.2', '8.3']
        symfony-version: ['6.3.*', '6.4.*']

    name: PHP ${{ matrix.php-version }} - Symfony ${{ matrix.symfony-version }}

    steps:
    - uses: actions/checkout@v3

    - name: Setup PHP
      uses: shivammathur/setup-php@v2
      with:
        php-version: ${{ matrix.php-version }}

    - name: Install Symfony ${{ matrix.symfony-version }}
      run: |
        composer require "symfony/framework-bundle:${{ matrix.symfony-version }}" --no-update
        composer update --prefer-dist --no-progress
```

📝 **Checkpoint 4** : Combien de jobs sont exécutés en parallèle ?

Partie 6 : Badges et notifications (10 min)

6.1 Ajout des badges

Dans `README.md` :

```markdown
```

```
# Symfony Contact App

![CI](https://github.com/[votre-username]/symfony-contact-app/workflows/CI/badge.svg)
[![codecov](https://codecov.io/gh/[votre-username]/symfony-contact-app/branch/main/graph/badge.svg)](https://cod

## Description
...
```

6.2 Notifications Slack (optionnel)

yaml

```yaml
    - name: Slack Notification
      if: failure()
      uses: 8398a7/action-slack@v3
      with:
        status: ${{ job.status }}
        text: 'CI Failed on ${{ github.ref }}'
        webhook_url: ${{ secrets.SLACK_WEBHOOK }}
```

Partie 7 : Pull Request workflow (20 min)

7.1 Création d'une feature branch

bash

```bash
git checkout -b feature/add-export develop
```

Ajoutez une petite fonctionnalité et créez une PR vers `develop`.

7.2 Protection de branches

Dans GitHub Settings → Branches :

- Add rule pour `main` et `develop`
- ✅ Require a pull request before merging
- ✅ Require status checks to pass
- ✅ Require branches to be up to date

7.3 Review automatique

Ajoutez dans `.github/workflows/pr-check.yml` :

yaml

```yaml
name: PR Check

on:
  pull_request:
    types: [opened, synchronize, reopened]

jobs:
  label:
    runs-on: ubuntu-latest
    steps:
    - uses: actions/labeler@v4
      with:
        repo-token: "${{ secrets.GITHUB_TOKEN }}"

  size:
    runs-on: ubuntu-latest
    steps:
    - uses: codelytv/pr-size-labeler@v1
      with:
        GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
        xs_label: 'size/xs'
        xs_max_size: 10
        s_label: 'size/s'
        s_max_size: 100
        m_label: 'size/m'
        m_max_size: 500
        l_label: 'size/l'
        l_max_size: 1000
        xl_label: 'size/xl'
```

📝 **Checkpoint 5** : Screenshot de la PR avec les checks passés

Synthèse et métriques

Tableau de bord CI

Remplissez ces métriques après plusieurs builds :

Métrique	**Valeur**
Temps moyen de build	___min
Taux de succès	___%
Couverture de code	___%
Issues PHPStan	___
Violations CS Fixer	___

Améliorations identifiées

1.
2.
3.

Workflow final complet

Votre `.github/workflows/ci.yml` final devrait contenir toutes les optimisations. Committez-le !

Rendu

Repository GitHub avec :

- ✅ Workflow CI complet et fonctionnel
- ✅ Au moins 5 tests passants
- ✅ Couverture > 50%
- ✅ Analyse statique configurée
- ✅ Cache optimisé
- ✅ README avec badges
- ✅ Une PR merged avec tous les checks

🎉 **Félicitations !** Vous avez mis en place une intégration continue professionnelle !